

Code Walkthrough

DNF

按照“上 放代码，下 作用 执 它是干什么的” 种方式，把主刷图代码从头到尾拆开

工作区： `D:\yolo\yolo11\dnf`

点文件： `main.py` `detector.py` `door_strategy.py` `minimap_astar.py`

`minimap_nav.py` `game.py`

充 明： `minimap_route_demo.py` 会 ， `olaplug` 只做 录 明，因为它不是当前主刷图流程真正执 的代码

份文档不是“概括介绍”，是“按代码段顺序往下”。你看到的每段，会先把原代码摆出来，再紧接它的作用、执行以及它在整套刷图代码中的位置。这样你就好像看老师做代码批注一样，一段一段往下。

先提一个常关的事实：当前这套刷图主线没有真正用 `olaplug` 的 `BindWindow()`。它其实依的是 `win32gui` 找窗口，`pyautogui` 截图，`pydirectinput` 发盘入。所以以后的点会放在“现在真实的主”上。

-
1. `main.py`：程序总入口
 2. `detector.py`：大画目标检测
 3. `door_strategy.py`：择分
 4. `minimap_astar.py`：A 星算法
 5. `minimap_nav.py`：小地图别与线生成
 6. `game.py`：打怪、拾取、移动、卡住恢复
 7. `minimap_route_demo.py`：单独演示小地图导航的测试本
 8. 录：为什么 `olaplug` 不属于当前主刷图执行

main.py

它是总入口 你可以把它看成“总 度器” 它 找窗口 截图 用
别器 用小地图导 器，再把 果交 动作控制器 `Game` 去执

1

main.py

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
@File      : main.py
@Desc      : Win32-based DNF capture loop
"""

from __future__ import annotations

import pathlib
import time
from typing import Optional, Tuple

import cv2
import numpy as np
import pyautogui
import win32con
import win32gui
from loguru import logger

from dnf.game import Game
from dnf.minimap_nav import MiniMapNavigator

temp = pathlib.PosixPath
pathlib.PosixPath = pathlib.WindowsPath

from dnf.detector import Detector

WINDOW_OFFSET_X = 10
WINDOW_OFFSET_Y = 10
WINDOW_TITLE_KEYWORD = "鎧領筈錯疼芽鐸囧+"
WINDOW_CLASS_NAMES = {"鎧領筈錯疼芽鐸囧+", "鎧領筈錯疼芽鐸囧+録淚柊涓涓栵"}

```

段代码先把整个主程序的库全导来 `cv2` 用来处理图像, `numpy` 用来承接截图数据, `pyautogui` 用来截图, `win32gui` 和 `win32con` 用来找和摆放游戏窗口, `Game` 动作执行, `MiniMapNavigator` 小地图导航, `Detector` YOLO 检测

`temp = pathlib.PosixPath` 和后的 `pathlib.PosixPath = pathlib.WindowsPath` 是个兼容性小技巧, 常是为了防止某些模型或依赖在 Windows 下加载路径时, 把路径当成了 Posix 风格然后定义的几个窗口常量, 是后续找 DNF 窗口和摆窗口位时用的“滤条件”和“摆放目标位”

是总入口文件的“基础施区” 没开始真正刷图, 但后所有会依赖导入的类和常量 你可以理解为程序开机后的准备工作

2 DNF

函数 `_find_dnf_window()`

```
def _find_dnf_window() -> int:
    candidates = []

    def _enum_windows(hwnd: int, _: object) -> None:
        if not win32gui.IsWindowVisible(hwnd):
            return

        title = win32gui.GetWindowText(hwnd).strip()
        class_name = win32gui.GetClassName(hwnd).strip()
        if not title and not class_name:
            return

        title_match = WINDOW_TITLE_KEYWORD in title
        class_match = class_name in WINDOW_CLASS_NAMES or WINDOW_TITLE_KEYWORD in
class_name
        if not title_match and not class_match:
            return

        try:
            left, top, right, bottom = win32gui.GetWindowRect(hwnd)
            area = max(0, right - left) * max(0, bottom - top)
        except win32gui.error:
            area = 0

        candidates.append((area, hwnd, title, class_name))

    win32gui.EnumWindows(_enum_windows, None)
    if not candidates:
        raise RuntimeError("婁℃沿錄惧垠 DNF 给耦彝")

    candidates.sort(key=lambda item: item[0], reverse=True)
    hwnd = candidates[0][1]
    logger.info(
        "緇戔畚给耦彝: hwnd={}, title={}, class={}",
        hwnd,
        candidates[0][2],
        candidates[0][3],
    )
    return hwnd
```

段代码的任务只有一个：在系统所有可窗口，把 DNF 窗口找出来，并返回它的句柄
hwnd 句柄你可以理解成“一个窗口在 Windows 系统的份号”

程序先创建一个空列 `candidates`，准备所有疑似 DNF 的窗口 然后用
`win32gui.EnumWindows()` 遍历所有窗口，每遍历到一个窗口，就调用函数
`_enum_windows()` 去检查 检查 序是：先看它是否可，再拿到窗口标和类名；如果标
和类名空，就；如果标和类名不匹配 DNF 关，也 只有疑似 DNF 的窗
口才 往下 接着它会 出窗口矩形大小，算 个 积，把 积 句柄 标 类名 存
入列

它的本不是“后台 定窗口”，是“从系统找出我 着哪 个游戏窗口” 最后代码
按 积从大到小排序，最大的 个候 窗口作为最 果， 样可以 免同时开多个相关窗
口时 到很小的子窗口

3

函数 `_place_window()`

```
def _place_window(hwnd: int) -> None:
    win32gui.ShowWindow(hwnd, win32con.SW_RESTORE)
    left, top, right, bottom = win32gui.GetWindowRect(hwnd)
    width = max(1, right - left)
    height = max(1, bottom - top)
    win32gui.SetWindowPos(
        hwnd,
        win32con.HWND_TOPMOST,
        WINDOW_OFFSET_X,
        WINDOW_OFFSET_Y,
        width,
        height,
        win32con.SWP_SHOWWINDOW,
    )
```

找到窗口之后，程序会把窗口恢复出来并移动到一个比 固定的位，也就是屏幕左上 的 `(10, 10)`

`ShowWindow(..., SW_RESTORE)` 的意思是如果窗口最小化了，就把它恢复 然后 取窗口当前的四个 界坐标，算出宽 最后 用 `SetWindowPos()` 把窗口放到 位，并 成 显示

步的目的，是 后 的截图区域更稳定 因为程序后 截图时依 窗口客户区的位，如果窗口乱 挡住 到任务栏，检测 果就会变得不稳定

4

函数 `_get_client_region()` 与 `_capture_client()`

```
def _get_client_region(hwnd: int) -> Tuple[int, int, int, int]:
    left_top = win32gui.ClientToScreen(hwnd, (0, 0))
    client_rect = win32gui.GetClientRect(hwnd)
    left = int(left_top[0])
    top = int(left_top[1])
    width = int(client_rect[2] - client_rect[0])
    height = int(client_rect[3] - client_rect[1])
    if width <= 0 or height <= 0:
        raise RuntimeError("鑾峰彌媿告堯淪(=)垲鍢哄け璐?)")
    return left, top, width, height

def _capture_client(hwnd: int) -> Tuple[np.ndarray, Tuple[int, int]]:
    left, top, width, height = _get_client_region(hwnd)
    img = pyautogui.screenshot(region=(left, top, width, height))
    img_np = np.array(img)
    return img_np, (width, height)
```

段代码 真正把游戏画 “拍下来” 注意它截的是窗口客户区，也就是游戏真正显示内容的区域，不是包括标 栏和 框的整个窗口

`ClientToScreen(hwnd, (0, 0))` 会把客户区左上 换算成屏幕坐标，`GetClientRect()` 会 回客户区的宽 两个 果组合 来，就 得到截图 的 `left, top, width, height` 然后 `pyautogui.screenshot(region=...)` 用 个矩形去截屏，截出来的是 个 PIL 图片，再 成 `numpy` 数组，后 才 `OpenCV` 和 `YOLO`

段就是 “ 程序眼 画 ” 如果没有截图，后 的检测器 导 器 动作判断就全 无从

5

函数 `main()` 的开头

```
def main() -> None:
    device_type = ""
    detector = Detector(device_type)
    navigator = MiniMapNavigator("generic")

    hwnd = _find_dnf_window()
    _place_window(hwnd)
```

主函数刚开始时，会先把核心工具对 准备好 `Detector` 是大画 检测器，`MiniMapNavigator` 是小地图导 器 然后它立刻去找 DNF 窗口并摆好位

`device_type = ""` 代 检测器 已判断使用 GPU 是 CPU 接着构 `Detector` 和 `MiniMapNavigator("generic")` 示当前 按 `generic` 地图 格来 小地图 准备好模型和导 器后，程序立刻 定到目标窗口并把窗口移动到固定位

是 大循环开始前的 “ 次性初始化” 模型只加 次，窗口只摆 次，后 每 帧只做截图 检测 判断和执 ，不会反复 复 些 准备动作

6

函数 `main()` 的核心前半段

```
while True:
    start_time = time.time()
    try:
        img_np, (width, height) = _capture_client(hwnd)
    except Exception as exc:
        logger.exception("贴图混浊接触: {}", exc)
        time.sleep(0.2)
        continue

    img, obj = detector.detect(img_np)
    logger.info("obj: {}", obj)

    frame_bgr = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)
    route_snapshot = navigator.build_route_snapshot(frame_bgr, obj or [])
    route_direction = (
        route_snapshot.next_room_direction.upper()
        if route_snapshot.next_room_direction
        else None
    )
```

是真正每帧在复执的核心前半段 它先截图，然后做大画面别，再做小地图导航，最后拿到个“下一步应该往哪”的方向提示

程序入死循环后，先记录开始时，方便每时每刻接着用 `_capture_client(hwnd)` 截一张当前游戏画面 如果截图失败，就记录异常，稍微休眠一下，直接开始下次截图成功后，把图像 `detector.detect()` 这个函数会返回两样东西：一张画好框的图 `img`，以及检测对象列表 `obj` 然后它把原始截图从 RGB 变成 BGR，`navigator.build_route_snapshot()` 导航器会合小地图模板匹配结果和目标检测结果，返回个线快照对象，有当前房目标房线方向等信息最后从 `next_room_direction` 取出，并成大写形式后的动作层用

你可以把这段理成“看图 + 算方向” 它本不按，也不打怪，只得出个：现在画面有哪些东西，当前副本线应该往哪个房

7

函数 `main()` 的核心后半段

```
    logger.info(
        "route: current={}, boss={}, query={}, elite={}, down={}, direction={},
door={}, scores={}",
        route_snapshot.current_room,
        route_snapshot.boss_room,
        route_snapshot.query_room,
        route_snapshot.elite_room,
        route_snapshot.down_room,
        route_direction,
        route_snapshot.selected_door_center,
        route_snapshot.debug_scores,
    )

    game = Game(obj, width, height, route_direction)
    game.run()

    display = cv2.resize(img, (640, 360))
    cv2.imshow("112233", display)
    logger.info("澶勫姳鏁楅櫨: {}", time.time() - start_time)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

是主循环的后半段：把刚刚得到的 别 果和 线方向交 动作执 器 `Game` , 它决定 帧到底 打怪 捡东 是往

先打印 大串 线 信息, 方便开发时看当前 别到的房 和方向 然后创建 `Game(obj, width, height, route_direction)`, 把检测对 画 宽 和房 级方向 交 它 接着 用 `game.run()` 执 动作 执 完后, 会把检测 果图 放显示到 个窗口 , 便于 眼 察程序 别得对不对 最后 录本 处理 时, 并允 用户按 `q` 出

段就是整个主 的“分发和收尾”：上游已 看懂画 , 下游 把 个判断变成 盘动作 可以 `main.py` 到 就完成了 己的 , 后 真正的 为控制就交 `Game` 了

detector.py

用 YOLO 模型 别大画 的玩家 怪物 Boss 币 掉 物 它
相当于主程序的第 只眼

1

类 `Detector` 的 `__init__`

```
class Detector:
    def __init__(self, device_type: str = ""):
        self.img_size = 640
        self.conf_thres = 0.75
        self.iou_thres = 0.45
        self.hide_labels = False
        self.hide_conf = False

        self.device = device_type or ("0" if torch.cuda.is_available() else "cpu")
        self.weights = str(Path(__file__).resolve().parent / "shzn.pt")
        self.model = YOLO(self.weights)
        self.names = self.model.names
        self.colors = [[random.randint(0, 255) for _ in range(3)] for _ in
                        range(len(self.names))]
```

段代码 把 YOLO 检测器真正准备好 它会 图像大小 信度 值 IOU 值, 然后
加 本地模型文件 `shzn.pt`

`img_size = 640` 示推理时把图像按 640 尺寸 模型 `conf_thres = 0.75` 示低于
0.75 信度的目标就不 了 `iou_thres = 0.45` 用来做 NMS 去 备 择 是: 如
果外 没强制指定 备, 就 动判断显卡是否可用, 可用就用 GPU 0, 不可用就 到 CPU 之
后用 `YOLO(self.weights)` 把模型 入内存, 再从模型 拿到类别名称 `self.names`, 最
后 每个类别 机生成 个画框

就是目标检测器的“开机步 ” 如果 段没 , 程序后 就根本不知 画 哪 是
怪 哪 是 哪 是玩家

2

两个 态工具函数

```
@staticmethod
def _normalize_label(label: str) -> str:
    label = str(label).strip().lower()
    if "door" in label:
        return "door"
    return label

@staticmethod
def _xyxy_to_xywh_top_left(xyxy: np.ndarray) -> List[float]:
    x1, y1, x2, y2 = [float(v) for v in xyxy]
    return [x1, y1, max(0.0, x2 - x1), max(0.0, y2 - y1)]
```

段不是做检测， 是 把模型吐出来的数据整理成后 更容易用的格式

`_normalize_label()` 会把类别名 成小写字符串 如果类别名字 包含 `door`，就
标准化成 `door` 样就算模型 存在不同 标签，后 动作层也可以 按 “ ” 处理
`_xyxy_to_xywh_top_left()` 则把 YOLO 常用的矩形框格式 `[x1, y1, x2, y2]` 成
`[x, y, w, h]` 的 `x, y` 是左上，`w, h` 是宽

本 上是在做“数据清洗和格式 ” 后 的 `Game` 导 器 策略 依 种 格
式，否则每个模块 得 己 新 析框数据

3

函数 `detect()` 的开头

```
def detect(self, img0: np.ndarray) -> Tuple[np.ndarray | None, list | None]:
    """
    检测怪物并返回 BGR 图像和置信度列表。
    - annotated_bgr: 包含怪物检测结果的 BGR 图像
    - obj: 包含怪物检测结果的字典列表，格式为 [{'monster': {'xywh': [x, y, w, h], 'conf': 0.84}}, ...]
    """
    if img0 is None:
        return None, None

    img_bgr = cv2.cvtColor(img0, cv2.COLOR_RGB2BGR)
```

这段代码是目标检测的真正入口。它接收一张截图，准备把它输入 YOLO 模型。

第一步先判断输入图像是不是空，如果是空图，就直接返回空。这样可以避免程序某帧截图丢失时崩掉。然后它把图片从 RGB 转换成 BGR。原因是前面的截图来自 `pyautogui`，通常是 RGB，而 OpenCV 和后面的显示习惯用 BGR。

它相当于把外部输入的原始截图，转换成符合 YOLO 和 OpenCV 处理的图像格式，为真正的模型推理做准备。

4 YOLO

函数 `detect()` 的模型 测 分

```
result = self.model.predict(
    source=img_bgr,
    imsz=self.img_size,
    conf=self.conf_thres,
    iou=self.iou_thres,
    device=self.device,
    verbose=False,
)[0]

annotator = Annotator(img_bgr.copy(), line_width=3, example=self.names)
obj = []
```

段代码真正 用了模型， 模型对当前 帧做目标检测

`self.model.predict(...)` 会 回 个 果列 ， 因为 次只推 张图，所以取第 个 果 `[0]` 传 去的参数正是初始化时 好的图像大小 信度 值 IOU 值和 备 `Annotator` 则是 个画框 助工具，它会把每个检测框和类别名画到图上，方便显示 `obj = []` 是准备收 后 整理好的 构化检测 果

从 开始，程序已 不是在“准备检测”， 是在“真正看图” 从 步开始，程序会 知 当前画 到底有没有玩家 怪物 和掉 物

5

函数 `detect()` 的后半段

```
if result.bboxes is not None and len(result.bboxes) > 0:
    boxes_xyxy = result.bboxes.xyxy.cpu().numpy()
    boxes_conf = result.bboxes.conf.cpu().numpy()
    boxes_cls = result.bboxes.cls.cpu().numpy().astype(int)

    for xyxy, conf, cls in zip(boxes_xyxy, boxes_conf, boxes_cls):
        raw_label = self.names[int(cls)]
        label = self._normalize_label(raw_label)
        xywh = self._xyxy_to_xywh_top_left(xyxy)
        conf = round(float(conf), 2)

        annotator.box_label(
            xyxy,
            raw_label if self.hide_conf else f"{raw_label} {conf:.2f}",
            color=tuple(self.colors[int(cls)]),
        )
        obj.append({label: {"xywh": xywh, "conf": conf}})

    return annotator.result(), obj
```

段代码把模型原始 出整理成后 各个模块更好消 的格式, 并且 手画框形成 图

先判断有没有检测框 如果有, 就把每个框的位 信度 类别 号 取出来, 并从 GPU 张 成 CPU 上的 `numpy` 数组 然后 个框处理: 先拿类别 号去类别名 找出原始标签, 再做标准化, 再把坐标格式从 `xyxy` 成 `xywh`, 最后把 信度保留两位小数

`annotator.box_label()` 用来把 个框画到图上, `obj.append({label: {"xywh": xywh, "conf": conf}})` 则把它存成 格式, 供后 的 使用

段的最 成果有两样: 张可 化后的图, 份 构化对 列 前 人看, 后 程序做决策用 主刷图 真正依 的是 个 `obj` 列

door_strategy.py

当 A 星告 程序 “下 房在右 ” 时, 个文件 在当前画 检测到 的多扇 , 挑出最 合往右 的 扇

1

文件

```
from dataclasses import dataclass
from typing import List, Optional, Sequence, Tuple

Point = Tuple[float, float]

@dataclass
class DoorCandidate:
    bbox: Sequence[float]
    center: Point

def opposite_direction(direction: Optional[str]) -> Optional[str]:
    mapping = {
        "left": "right",
        "right": "left",
        "up": "down",
        "down": "up",
    }
    return mapping.get(direction)
```

段先定义 候 的数据 构，再定义 个方向取反函数 候 只存两样最关 的数
据：包围框和中心点

`DoorCandidate` 是 个简单的数据类，后 导 器会把所有检测到的 包 成 种对
`opposite_direction()` 则用来 算某个方向的反方向，比如往左的反方向是右，往上的反
方向是下

个文件后 的 分 ， 知 上 的方向和它的反方向，好在挑 时减少 “掉头回
” 的概率 所以 先把基础工具定义出来

2

函数 `_axis_offsets()` 与 `_direction_match()`

```
def _axis_offsets(player: Point, door: Point) -> Tuple[float, float]:
    dx = door[0] - player[0]
    dy = door[1] - player[1]
    return dx, dy

def _direction_match(expected_direction: str, dx: float, dy: float, margin: float) -> bool:
    if expected_direction == "right":
        return dx > margin
    if expected_direction == "left":
        return dx < -margin
    if expected_direction == "up":
        return dy < -margin
    if expected_direction == "down":
        return dy > margin
    return True
```

段代码 回答两个：第一，某扇 相对玩家偏在哪；第二，扇 算不算符合期望方向

`_axis_offsets()` 很简单，就是拿 中心减去玩家中心，得到 `dx` 和 `dy` 然后 `_direction_match()` 根据期望方向去判断 例如期望往右 时，只有 `dx > margin` 才算真正符合“右”，也就是 必 明显在玩家右，不是 强 右 点点

的 `margin` 就是 种容差 因为 的位 不可 永 刚好标准对，如果不留 个余，就容易把斜方向或 得很 的 判成目标

3

函数 `choose_best_door()` 的前半段

```
def choose_best_door(
    doors: Sequence[DoorCandidate],
    player_center: Point,
    expected_direction: Optional[str],
    last_direction: Optional[str] = None,
    margin: float = 16.0,
) -> Optional[DoorCandidate]:
    if not doors:
        return None
    if expected_direction is None:
        return min(doors, key=lambda item: abs(item.center[0] - player_center[0]) +
abs(item.center[1] - player_center[1]))
```

段是 择函数的 底 步 先处理 些简单情况, 免后 复杂 分没必 地执

如果 列 为空, 函数直接 回 `None`, 示当前没有可 如果有 但外 没有 出期望 方向, 它就 求其次, 直接 离玩家最 的 用的是曼哈 离形式的 似值, 也就是 `abs(dx) + abs(dy)`

它保 了 策略在“没有 线提示”时也 工作 也就是, 哪 A 星暂时没算出方向, 程序也不会完全失去 动 力

是整份 策略 最关 的地方 它会 每 扇 打分，然后区分成“方向匹 候 ”和“普 底候 ”两类

程序先算出上 方向的反方向，作为后 反向惩 的依据 然后对每扇 ， 算 相对玩家的 横纵偏移 `match` 示 扇 是否满 期望方向，例如往右时 是否明显在右 如果期望方向是左右， 么 `forward` 就看左右前 ， `sideways` 看上下偏移；如果期望方向是上下， 则相反 之后它 会根据上 方向的反方向，加 个 常大的 `reverse_penalty`，用来减少 程序刚刚往左，下 瞬 又 上掉头往右的情况 最 分数 `score` 是 个三元组：第 优先 看前 值，第二优先看侧向偏移，第三优先看总 离

本 上 是 个启发式 分算法，不是深度学习 它的目标是把“房 级方向” 成“画 级 择”，也就是把 A 星 出的抽 果变成现实可 的 扇

5

函数 `choose_best_door()` 的 尾

```
if candidates:
    candidates.sort(key=lambda item: item[0])
    return candidates[0][1]

fallback.sort(key=lambda item: item[0])
return fallback[0][1]
```

段代码 从前 打好分的 ，真正 出最后 扇

如果有“方向匹 ”的候 ，就按分数排序，拿分数最优的 扇 如果 个 没有， 明当前画 没有明显满 目标方向的 ， 就 回到 `fallback` 列 ，从所有 最合理的 扇

是 策略的最 决策出口 它保 了“优先尊 线方向，但找不到完全匹 的 时，也 尽 个最像的 ”，从 免程序卡死在 择 步

minimap_astar.py

它 房 级 径 划 注意是“房 级”，不是“像素级” 它的 出
不是 下怎么 步 步 弯， 是“下 房应 往上 下 左 右哪 ”

1

文件

```
import heapq
from dataclasses import dataclass
from typing import Iterable, List, Optional, Sequence, Tuple

GridPoint = Tuple[int, int]

@dataclass(order=True)
class _Node:
    f: int
    position: GridPoint
    g: int = 0
    h: int = 0
    parent: Optional["_Node"] = None

def heuristic(a: GridPoint, b: GridPoint) -> int:
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

段定义了 A 星算法最基础的数据 构： 点，以及估价函数 `heuristic()`

`_Node` 有五个核心字段：`position` 是当前格子坐标，`g` 是从 点 到 的真实代价，`h` 是从 到 点的估 代价，`f` 是总估价，也就是 `g + h`，`parent` 是当前 点的父 点，后 回 径时 用 `heuristic()` 用的是曼哈 离，因为当前地图只 上下左右四 ，不 斜

是 A 星算法的地基 没有 点 构，就没法 护 径搜索状态；没有启发函数，A 星也就化成普 的广搜或 Dijkstra 格搜索了

2

函数 `_neighbors()`

```
def _neighbors(position: GridPoint, grid: Sequence[Sequence[int]], priority: str) ->
    Iterable[GridPoint]:
    x, y = position
    directions = {
        "right": ((0, 1), (1, 0), (-1, 0), (0, -1)),
        "left": ((0, -1), (-1, 0), (1, 0), (0, 1)),
        "up": ((-1, 0), (0, -1), (0, 1), (1, 0)),
        "down": ((1, 0), (0, 1), (0, -1), (-1, 0)),
        "default": ((0, 1), (1, 0), (-1, 0), (0, -1)),
    }
    candidates = directions.get(priority, directions["default"])
    for dx, dy in candidates:
        nx = x + dx
        ny = y + dy
        if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]) and grid[nx][ny] == 0:
            yield (nx, ny)
```

段代码 从当前房 出发, 找出 到 上下左右 居房

程序先根据当前优先方向 `priority`, 决定 居界检查 序 比如优先方向是 `right`, 就先看右, 再看下 上 左 然后它 个尝 些偏移, 算新坐标 `(nx, ny)` 只有当新坐标 没有 界, 且 `grid[nx][ny] == 0` 时, 才把 个 居 回出去 界 0 示可 , 0 示 碍

段既控制 " 不 " , 也控制 " 同样可 时优先往哪 尝 " 所以它不只是列出 居, 是 A 星 线偏好 个 味器

3 A

函数 `a_star()` 的开头

```
def a_star(grid: Sequence[Sequence[int]], start: GridPoint, end: GridPoint, priority:
str = "right") -> Optional[List[GridPoint]]:
    if start == end:
        return [start]

    open_list: List[_Node] = []
    start_node = _Node(f=0, position=start, g=0, h=heuristic(start, end), parent=None)
    start_node.f = start_node.g + start_node.h
    heapq.heappush(open_list, start_node)
    closed = set()
    best_g = {start: 0}
```

段代码 把 A 星搜索的初始状态搭 来

如果 点和 点本来就是同 个房 , 直接 回只包含 点的 径 否则, 创建开放列
`open_list`, 把 点包 成 个 点压 去 点的真实代价 `g` 为 0, 估 代价 `h` 由曼哈
离 出, 总分 `f = g + h` `closed` 合用于 录已 确 处理 的点, `best_g` 用
来 录某个位 目前找到的最优真实代价, 免反复 更差的 径

步相当于在地图上摆下第 个探索点, 并准备好之后 用的 “已处理 合” 和 “最优代价
”

4 A

函数 `a_star()` 的主体

```
while open_list:
    current = heapq.heappop(open_list)
    if current.position in closed:
        continue
    closed.add(current.position)

    if current.position == end:
        path: List[GridPoint] = []
        node: Optional[_Node] = current
        while node is not None:
            path.append(node.position)
            node = node.parent
        return list(reversed(path))

    for next_pos in _neighbors(current.position, grid, priority):
        next_g = current.g + 1
        if next_g >= best_g.get(next_pos, 10 ** 9):
            continue
        best_g[next_pos] = next_g
        next_h = heuristic(next_pos, end)
        heapq.heappush(
            open_list,
            _Node(
                f=next_g + next_h,
                position=next_pos,
                g=next_g,
                h=next_h,
                parent=current,
            ),
        )
return None
```

段就是 A 星算法真正来的地方 它会持 从开放列 拿出最优 点, 不断向外扩展, 直到找到 点

程序每次 从堆 弹出 `f` 值最小的 点作为当前 点 如果 个位 之前已 处理 , 就 否则把它加入 `closed` 如果当前 点就是 点, 么程序就沿着 `parent` 往回 , 把整条 径倒着拼出来, 再反 成从 点到 点的 序 如果不是 点, 就去拿它的 居 对于每个 居, 真实代价 `next_g` 就是当前 `g + 1` 如果 个代价不比之前 录的更 优, 就不 它; 如果更优, 就更新 `best_g` , 新 算估价 `h` , 再把新 点压入堆

就是整套房 级寻 的核心算法 它不会直接告 在屏幕 怎么 , 但会告 程序从当前房 出发, 最短 线应 哪些房

5

函数 `judge_direction()` 与 `next_direction()`

```
def judge_direction(starting_point: GridPoint, end_point: GridPoint) -> Optional[str]:
    x1, y1 = starting_point
    x2, y2 = end_point
    if x2 > x1:
        return "down"
    if x2 < x1:
        return "up"
    if y2 > y1:
        return "right"
    if y2 < y1:
        return "left"
    return None

def next_direction(grid: Sequence[Sequence[int]], start: GridPoint, end: GridPoint,
priority: str = "right") -> Optional[str]:
    path = a_star(grid, start, end, priority=priority)
    if not path or len(path) < 2:
        return None
    return judge_direction(path[0], path[1])
```

段代码把 A 星整条路径，步压成当前最好的信息：下一步应往哪

`next_direction()` 先用前的 `a_star()` 算出完整路径 如果路径为空，或路径长度不 2，说明当前无法前进，就回空 否则只看路径的前两个房：当前房和下一个房，然后交 `judge_direction()` 去比它们的列变化，从得出 `up` `down` `left` `right` 之

是 A 星和整个刷图系统接的关 出口 动作层不 整条路径，它只知道“下一个房在哪个方向”，然后再去当前画 找对应方向的

minimap_nav.py

是小地图导航中心，它会输出小地图区域，用模板匹配当前房间的Boss、房间、精英房等，再用A*算法算出下一步方向，并尝试根据方向找到最合适的

1

文件 数据类和地图

```
from copy import deepcopy
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional, Sequence, Tuple

import cv2
import numpy as np

from dnf.door_strategy import DoorCandidate, choose_best_door
from dnf.minimap_astar import next_direction

Point = Tuple[int, int]
Rect = Tuple[int, int, int, int]

@dataclass
class MapSpec:
    name: str
    crop_rect_1067: Rect
    minimap_width: int
    minimap_height: int
    rows: int
    cols: int
    room_grid: List[List[int]]

@dataclass
class RouteSnapshot:
    current_room: Optional[Point]
    boss_room: Optional[Point]
    query_room: Optional[Point]
    elite_room: Optional[Point]
    down_room: Optional[Point]
    next_room_direction: Optional[str]
    selected_door_center: Optional[Tuple[float, float]]
    debug_scores: Optional[Dict[str, float]] = None
```



2

函数 `_all_walkable()` 与 `MAP_SPECS`

```
def _all_walkable(rows: int, cols: int) -> List[List[int]]:
    return [[0 for _ in range(cols)] for _ in range(rows)]

MAP_SPECS: Dict[str, MapSpec] = {
    "generic": MapSpec(
        name="generic",
        crop_rect_1067=(893, 52, 1055, 142),
        minimap_width=162,
        minimap_height=90,
        rows=5,
        cols=9,
        room_grid=_all_walkable(5, 9),
    ),
    "haibolun": MapSpec(
        name="haibolun",
        crop_rect_1067=(966, 52, 1056, 124),
        minimap_width=90,
        minimap_height=72,
        rows=4,
        cols=5,
        room_grid=_all_walkable(4, 5),
    ),
}
```

定义了不同小地图模板的基本参数，并 每 种地图生成 个“全可 ”的房 格

`_all_walkable()` 会生成 个全 0 的二 数组，其中 0 示 房 位 可 然后
`MAP_SPECS` 列出不同地图名对应的 ，比如 `generic` 和 `haibolun` 每个 包含
小地图在 1067 宽度画 中的 剪区域，以及 个地图应 划分成多少 多少列

步是在告 导 器：“不同地图 得不 样，所以你 图的位 格 列数 房 模
知 ”当前代码 的房 格没有真实 碍信息，基本是假 整个 格 可以 ，
也是 份实现的 个局

3

类 `MiniMapNavigator` 的 `__init__`

```
class MiniMapNavigator:
    def __init__(self, map_name: str = "generic", assets_dir: Optional[Path] = None,
threshold: float = 0.7):
        if map_name not in MAP_SPECS:
            map_name = "generic"
        self.map_name = map_name
        self.spec = MAP_SPECS[map_name]
        self.threshold = threshold
        self.assets_dir = assets_dir or (Path(__file__).resolve().parent / "res")
        self.templates = {
            "hero": self._load_template("map_hero.png"),
            "boss": self._load_template("map_boss.png"),
            "query": self._load_template("map_query.png"),
            "query1": self._load_template("map_query1.png"),
            "query3": self._load_template("map_query3.png"),
            "query4": self._load_template("map_query4.png"),
            "query5": self._load_template("map_query5.png"),
            "elite": self._load_template("map_elite.png"),
            "special": self._load_template("map_special_room.png"),
        }
        down_template = self._load_template_optional("map_down.png")
        if down_template is not None:
            self.templates["down"] = down_template
        self.last_direction: Optional[str] = None
```

段代码 创建 一个可用的小地图导 器，并把所有小图标模板 内存

先确 传入的地图名称是否存在，如果不存在就 回到 generic 然后 录当前帧 的出
图 格和 模板匹 值 指向当前文件目录下的 文件夹 吧
图标 Boss 图标 号房图标 精 房图标 特殊房图标等全 来 模板是
的，所以单独用 个“可 加 ”函数 最后 last_direction 用来 住上 次 线方
向，方便后 时减少掉头

步相当于 导 器准备好 “ 别字典” 之后每次看到小地图，程序 是拿 些模板 张
张去比对，判断图标在哪

4

函数 `_load_template()` 与 `_load_template_optional()`

```
def _load_template(self, filename: str) -> np.ndarray:
    path = self.assets_dir / filename
    image = cv2.imread(str(path), cv2.IMREAD_COLOR)
    if image is None:
        raise FileNotFoundError(f"template not found: {path}")
    return image

def _load_template_optional(self, filename: str) -> Optional[np.ndarray]:
    path = self.assets_dir / filename
    if not path.exists():
        return None
    image = cv2.imread(str(path), cv2.IMREAD_COLOR)
    return image
```

两段函数 把小地图模板文件从磁盘 来

`_load_template()` 会严格 求模板必 存在, 如果图片没 到, 就直接报 , 因为 明
主导 少关 模板 `_load_template_optional()` 则更宽松 些, 如果文件不存在就
回 `None` , 不会中断程序

区分 “必 模板” 和 “可 模板” , 体现的是 种稳定性 : 主流程必 依 的模板不
丢, 可 力丢了可以 化

5

函数 `_scaled_crop_rect()` 与 `extract_minimap()`

```
def _scaled_crop_rect(self, frame: np.ndarray) -> Rect:
    frame_h, frame_w = frame.shape[:2]
    x1, y1, x2, y2 = self.spec.crop_rect_1067
    scale_x = frame_w / 1067.0
    scale_y = frame_h / 600.0
    return (
        int(round(x1 * scale_x)),
        int(round(y1 * scale_y)),
        int(round(x2 * scale_x)),
        int(round(y2 * scale_y)),
    )

def extract_minimap(self, frame: np.ndarray) -> np.ndarray:
    x1, y1, x2, y2 = self._scaled_crop_rect(frame)
    return frame[y1:y2, x1:x2]
```

段 从整张游戏截图中，把小地图准确 出来

地图 存的 剪区域是基于 1067×600 类基准尺寸写的，但真实截图不 定总是 个大 小 所以程序先拿当前截图的宽 ，和基准宽 比例换算， 算出当前分 率下的小地图 区域 之后 `extract_minimap()` 根据换算出来的坐标直接把 块图 剪出来

步本 上是在做分 率 否则 旦游戏窗口大小略有变化，程序就会 位 ，小地图 别就全乱了

6

函数 `_match_template()` 与 `_match_template_debug()`

```
def _match_template(self, image: np.ndarray, template: np.ndarray, threshold:
Optional[float] = None) -> List[Tuple[float, float]]:
    threshold = self.threshold if threshold is None else threshold
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
    result = cv2.matchTemplate(image_gray, template_gray, cv2.TM_CCOEFF_NORMED)
    ys, xs = np.where(result >= threshold)
    matches: List[Tuple[float, float]] = []
    h, w = template_gray.shape[:2]
    for y, x in zip(ys, xs):
        matches.append((x + w / 2.0, y + h / 2.0))
    return matches

def _match_template_debug(self, image: np.ndarray, template: np.ndarray) ->
Tuple[List[Tuple[float, float]], float]:
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
    result = cv2.matchTemplate(image_gray, template_gray, cv2.TM_CCOEFF_NORMED)
    _, max_val, _, _ = cv2.minMaxLoc(result)
    ys, xs = np.where(result >= self.threshold)
    matches: List[Tuple[float, float]] = []
    h, w = template_gray.shape[:2]
    for y, x in zip(ys, xs):
        matches.append((x + w / 2.0, y + h / 2.0))
    return matches, float(max_val)
```

是小地图 别的核心：模板匹 程序拿 张小图标，在小地图上滑动比 ，看哪 最像它

先把大图和模板 成灰度图，减少 差异带来的影响 然后 用 `cv2.matchTemplate(..., TM_CCOEFF_NORMED)` 做归 化相关性匹 ，得到 张相似度热力图 程序会找出所有相似度大于 值的位 ，再把 些位 换算成模板中心点 版的函数则 外取出全局最大相似度 `max_val` ，方便你判断某个模板当前到底匹 得好不好

就是导 器的“第二只眼 ” 它不 别大画 的怪和 ， 是专 别 UI 级别的小图标，比如玩家当前位 图标 Boss 图标 号房图标

7

函数 `compute_room_id()`

```
def compute_room_id(self, x: float, y: float, minimap_image: np.ndarray) -> Point:
    height, width = minimap_image.shape[:2]
    cell_w = width / self.spec.cols
    cell_h = height / self.spec.rows
    row = int(y // cell_h)
    col = int(x // cell_w)
    row = max(0, min(self.spec.rows - 1, row))
    col = max(0, min(self.spec.cols - 1, col))
    return row, col
```

前 模板匹 只知 “图标在小地图 的哪个像素位 ” , 段 把 个像素位 换成 “第几 第几列的房 ”

程序先算出小地图每 个房 格子的宽和 然后用图标中心点 以格子尺寸, 得到它 在哪 哪 列 最后再用 `max` 和 `min` 把 果夹住, 止因为 差 界

是把 “图像空 ” 换成 “副本 格空 ” 的关 步 A 星吃的不是像素坐标, 是房 坐标, 所以必 层换算

8

函数 detect_room_markers()

```
def detect_room_markers(self, frame: np.ndarray) -> Dict[str, Optional[Point]]:
    minimap = self.extract_minimap(frame)
    current_room = None
    boss_room = None
    query_room = None
    elite_room = None
    down_room = None

    hero_matches = self._match_template(minimap, self.templates["hero"])
    if hero_matches:
        current_room = self.compute_room_id(*hero_matches[0], minimap)

    boss_matches = self._match_template(minimap, self.templates["boss"])
    if boss_matches:
        boss_room = self.compute_room_id(*boss_matches[0], minimap)

    query_matches = self._match_template(minimap, self.templates["query"])
    if not query_matches:
        query_matches = self._match_template(minimap, self.templates["query1"])
    if not query_matches:
        query_matches = self._match_template(minimap, self.templates["query3"])
    if not query_matches:
        query_matches = self._match_template(minimap, self.templates["query4"])
    if not query_matches:
        query_matches = self._match_template(minimap, self.templates["query5"])
    if query_matches:
        query_room = self.compute_room_id(*query_matches[0], minimap)

    elite_matches = self._match_template(minimap, self.templates["elite"])
    if elite_matches:
        elite_room = self.compute_room_id(*elite_matches[0], minimap)

    special_matches = self._match_template(minimap, self.templates["special"])
    if special_matches and elite_room is None:
        elite_room = self.compute_room_id(*special_matches[0], minimap)

    down_template = self.templates.get("down")
    if down_template is not None:
        down_matches = self._match_template(minimap, down_template,
threshold=0.62)
        if down_matches:
            down_room = self.compute_room_id(*down_matches[0], minimap)

    return {
        "current_room": current_room,
        "boss_room": boss_room,
        "query_room": query_room,
        "elite_room": elite_room,
```

```
"down_room": down_room,  
}
```

段代码把整张小地图上的 房 全 别出来，包括当前房 Boss 房 号房 精 房 和下层入口房

先从整张截图 出小地图 然后 类模板去匹 图标用来确定当前房 ，Boss 图标对应 Boss 房 号房比 特殊，代码准备了多种不同样式的模板，只 匹 到其中 种就 作 号房 精 房先尝 常 模板，如果没有匹 到，就用特殊房模板 底 下层入口则 用 个单独 低 点的 值 0.62 去 别，因为 类图标可 更 匹 最后把所有 别 果作为 个字典 回

步是在告 程序：“我现在 处哪 房，值得去的目标房分别在哪 ”后 目标房的 择和 A 星的 点， 段提供信息

9

函数 `get_debug_scores()` 与 `_door_candidates_from_objects()`

```
def get_debug_scores(self, frame: np.ndarray) -> Dict[str, float]:
    minimap = self.extract_minimap(frame)
    scores: Dict[str, float] = {}
    for name, template in self.templates.items():
        _, max_val = self._match_template_debug(minimap, template)
        scores[name] = round(max_val, 4)
    return scores

def _door_candidates_from_objects(self, objects: Sequence[dict]) ->
List[DoorCandidate]:
    doors: List[DoorCandidate] = []
    for item in objects:
        if "door" not in item:
            continue
        xywh = item["door"]["xywh"]
        if len(xywh) != 4:
            continue
        x, y, w, h = xywh
        if w <= 0 or h <= 0:
            continue
        center = (x + w / 2.0, y + h / 2.0)
        doors.append(DoorCandidate(bbox=xywh, center=center))
    return doors
```

段 半是 开发 看的 工具， 半是 策略做准备的数据 换

`get_debug_scores()` 会对每个模板 版匹 函数，并 录最大匹 分值 样开
发时可以直接看到，是不是某种模板现在分数太低， 换图或 值
`_door_candidates_from_objects()` 则 历 YOLO 检测 果，把所有 对 挑出来， 换
成 的 `DoorCandidate` 列 ， 带 滤掉格式异常或宽 无效的框

前 帮助你排查“小地图为什么没 别出来”，后 帮助你把“大画 检测出的 ”交
分算法 两件事共同服务于导 器的最 决策

10

函数 `_pick_target_room()`

```
def _pick_target_room(
    self,
    current_room: Optional[Point],
    query_room: Optional[Point],
    down_room: Optional[Point],
    elite_room: Optional[Point],
    boss_room: Optional[Point],
    prefer_special_room: bool,
) -> Optional[Point]:
    candidates: List[Optional[Point]] = []
    if prefer_special_room:
        candidates.extend([query_room, down_room, elite_room, boss_room])
    else:
        candidates.append(boss_room)

    for room in candidates:
        if room is None:
            continue
        if current_room is not None and room == current_room:
            continue
        return room
    return None
```

段代码 在多个候 目标房中， 出当前 最值得前往的 个房

如果 `prefer_special_room` 为真，候 序就是 号房 下层房 精 房 Boss 房 也就是，它优先去特殊房，再去 Boss 不偏好特殊房，就只 Boss 房 之后它按 序 历候 列，空值和当前所在房，只 找到第 个可用目标，就直接 回

其实是副本 线策略的体现 A 星只会算“从哪到哪怎么”，但“点到底 哪 房”件事，是 段目标房 择 决定的

11

A

函数 `build_route_snapshot()` 的前半段

```
def build_route_snapshot(
    self,
    frame_bgr: np.ndarray,
    detection_objects: Sequence[dict],
    prefer_special_room: bool = True,
) -> RouteSnapshot:
    room_info = self.detect_room_markers(frame_bgr)
    current_room = room_info["current_room"]
    boss_room = room_info["boss_room"]
    query_room = room_info["query_room"]
    elite_room = room_info["elite_room"]
    down_room = room_info["down_room"]

    target_room = self._pick_target_room(
        current_room=current_room,
        query_room=query_room,
        down_room=down_room,
        elite_room=elite_room,
        boss_room=boss_room,
        prefer_special_room=prefer_special_room,
    )

    next_room_direction = None
    if current_room is not None and target_room is not None:
        next_room_direction = next_direction(deepcopy(self.spec.room_grid),
        current_room, target_room, priority="right")
```

段代码开始把前 所有小地图 别 果整合 来, 真正得出 “下 房应 往哪 ”

它先 用 `detect_room_markers()` 取当前房 Boss 房 号房等位 然后 用 `_pick_target_room()` 定目标房 接下来, 只 当前房 和目标房 存在, 就 用 `next_direction()`, 把当前房 和目标房丢 A 星, 它 回下 步方向 用 `deepcopy(self.spec.room_grid)` 是为了 止后 算法在 格上做改动时影响原始

段就是 “小地图 别” 和 “A 星寻 ” 的 合点 也正是在 , 程序第 次把 信息 成了 个真正可执 的方向提示

12

函数 `build_route_snapshot()` 的后半段

```
selected_door_center = None
if next_room_direction:
    player_center = None
    for item in detection_objects:
        if "player" in item:
            x, y, w, h = item["player"]["xywh"]
            player_center = (x + w / 2.0, y + h / 2.0)
            break
    if player_center is not None:
        selected_door = choose_best_door(
            self._door_candidates_from_objects(detection_objects),
            player_center=player_center,
            expected_direction=next_room_direction,
            last_direction=self.last_direction,
        )
        if selected_door is not None:
            selected_door_center = selected_door.center
            self.last_direction = next_room_direction

debug_scores = self.get_debug_scores(frame_bgr)
return RouteSnapshot(
    current_room=current_room,
    boss_room=boss_room,
    query_room=query_room,
    elite_room=elite_room,
    down_room=down_room,
    next_room_direction=next_room_direction,
    selected_door_center=selected_door_center,
    debug_scores=debug_scores,
)
```

段在 A 星得到房 方向之后， 把 个方向和大画 的 关 来，尝 找出当前画中最合 的 扇

如果 A 星已 出了方向， 程序就先从检测 果 找玩家的中心点 找到后，再把所有 整理成 `DoorCandidate` 列 ，然后 用 `choose_best_door()` 如果 成功，就把 的中心点 录到 `selected_door_center` ，并把当前方向保存成 `last_direction` 最后再 上份模板匹 分数，把所有信息打包成 个 `RouteSnapshot` 回

步 明：A 星并不是孤立存在的 它的方向 果会 步影响当前房 的 择 然当前主流程最后没有直接把 个 坐标传 `Game` 使用，但 已 完成了“房 方向 -> 方向”的 次关

game.py

是整套刷图 为的控制中心 它决定 帧到底应 打怪 捡东 朝
哪扇 移动 何时判断 己卡住 以及卡住后怎么恢复 你可以把它理 成“手和 ”

1

类 Game 的开头

```
class Game:
    _action_cache: Optional[str] = None
    _player: Optional[BoxDict] = None
    _pre_player: Optional[BoxDict] = None
    _index = 0
    _player_missing_count = 0
    _position_history: List[Tuple[float, float]] = []
    _stuck_count = 0
    _last_recover_time = 0.0
    _recover_step = 0
    _last_player_center: Optional[Tuple[float, float]] = None
    _room_entry_position: Optional[Tuple[float, float]] = None
    _room_entry_time = 0.0
    _flow_direction: Optional[str] = None
    _missing_player_recover_threshold = 10
    _route_search_direction: Optional[str] = None
    _route_search_phase = 0
    _route_search_start: Optional[Tuple[float, float]] = None

    def __init__(self, obj: Sequence[dict], width: int, height: int, direction):
        self._obj = obj or []
        self._width = width
        self._height = height
        self._player_xywh: Optional[List[float]] = None
        self._attack_x = 100
        self._attack_y = 100
        self._move_x = 20
        self._move_y = 20
        self._direction = direction
```

段代码先把动作层的所有“记忆变”和“值参数”准备出来

类变量分用于帧记忆状态，比如上帧按着哪个方向上帧玩家位置最几帧位历史入房的时卡住恢复到第几步等实例初始化分则接收本检测结果画面宽和线方向，并攻击围和拾取围值

段相当于动作系的“记忆仓库”因为刷图不是帧就束，是很多帧组成的为，所以必有帧状态，程序才知道己是不是卡住了刚从哪扇来上是不是在朝某个方向移动

2

工具函数 `_current_door_direction()` `_get_cls()` `_get_clss()`

```
def _current_door_direction(self) -> Optional[str]:
    if isinstance(self._direction, (list, tuple)):
        if not self._direction:
            return None
        if Game._index >= len(self._direction):
            return str(self._direction[-1]).upper()
        return str(self._direction[Game._index]).upper()
    if isinstance(self._direction, str) and self._direction.strip():
        return self._direction.strip().upper()
    return None

def _get_cls(self, cls_name: str) -> Optional[BoxDict]:
    for item in self._obj:
        if cls_name in item:
            return item[cls_name]
    return None

def _get_clss(self, cls_name: str) -> List[BoxDict]:
    result = []
    for item in self._obj:
        if cls_name in item:
            result.append(item[cls_name])
    return result
```

几个函数是动作层最常用的工具函数。一个从导航果拿方向提示，另外两个从YOLO检测列拿目标对。

`_current_door_direction()` 兼容两种输入：如果传来的方向是一个列表，它会根据 `Game._index` 取当前应走的步；如果是一个字符串，就直接回大写版本。
`_get_cls()` 只取第一个匹配某类的对，`_get_clss()` 会把所有同类对收集来，例如所有怪物、所有币、所有

本质上是在动作层提供数据接口，后的战斗拾取和用同一套方式去拿目标。

3

工具函数 `_release_direction_keys()` `_release_cached_action()` `_distance()`

```
def _release_direction_keys(self) -> None:
    for key in ("up", "down", "left", "right"):
        pydirectinput.keyUp(key)

def _release_cached_action(self) -> None:
    if not Game._action_cache:
        self._release_direction_keys()
        return
    for action in Game._action_cache.strip().split("_"):
        pydirectinput.keyUp(action.lower())
    self._release_direction_keys()
    Game._action_cache = None

def _distance(self, a: Tuple[float, float], b: Tuple[float, float]) -> float:
    return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5
```

段代码 两个很基础但 常 的 力：第 ，安全地松开当前按着的方向 ；第二，算两个点之 的直线 离

`_release_direction_keys()` 会把上下左右四个方向 松开
`_release_cached_action()` 更 些，它先根据 存动作字符串，比如 `RIGHT_UP`，把对应的 全 抬 ，然后再 用 次总 放，最后把 存动作清空 `_distance()` 则 回标准欧几 得 离，后 判断 “ 是否 入口” “ 是否突然 到另 个位 ” 时 用到

类工具函数 然不显眼，但 常关 动化程序最 “ 按住不放” 或 “前 帧的动作影响下 帧”，所以 放按 必 够稳

4

函数 `_infer_flow_direction_from_spawn()` 与 `_maybe_update_room_entry()`

```
def _infer_flow_direction_from_spawn(self, player_center: Tuple[float, float]) -> Optional[str]:
    x, y = player_center
    edge_scores = {
        "LEFT": x,
        "RIGHT": self._width - x,
        "UP": y,
        "DOWN": self._height - y,
    }
    nearest_edge = min(edge_scores, key=edge_scores.get)
    opposite = {
        "LEFT": "RIGHT",
        "RIGHT": "LEFT",
        "UP": "DOWN",
        "DOWN": "UP",
    }
    if edge_scores[nearest_edge] > min(self._width, self._height) * 0.28:
        return None
    return opposite[nearest_edge]

def _maybe_update_room_entry(self) -> None:
    if not self._player_xywh:
        return
    player_center = (self._player_xywh[0], self._player_xywh[1])
    previous = Game._last_player_center
    Game._last_player_center = player_center
    if previous is None:
        Game._room_entry_position = player_center
        Game._room_entry_time = time.time()
        Game._flow_direction =
self._infer_flow_direction_from_spawn(player_center)
        return

    jump_distance = self._distance(player_center, previous)
    if jump_distance >= 150:
        Game._room_entry_position = player_center
        Game._room_entry_time = time.time()
        Game._flow_direction =
self._infer_flow_direction_from_spawn(player_center)
        logger.info(
            "room transition inferred, entry={}, flow_direction={}",
            Game._room_entry_position,
            Game._flow_direction,
        )
```

段代码的任务是推断 刚 入 个新房 时，是从哪个 出现的，从 反推出副本推的大 流向

先看 离屏幕四个 哪个最 如果 离左 最 ， 常意味着他是从左侧 来的，因此接下来的推 方向大概率是往右 代码用 个 0.28 的比例 值判断 是否真的 ，不 就不强 推断 `_maybe_update_room_entry()` 则会在每 帧更新玩家中心点，并对比上 帧位 如果第 次 录玩家位 ，就直接把当前位 成入口；如果和上 帧 离突然 了 150 像素以上，就 为可 刚刚换房，于是更新入口点 更新时 和推断出来的流向

步很 ，因为后 为了 止 刚 房就回头 ，会用到“入口保护”和“反向 滤” 它的核心思想是：程序 住 己是从哪 来的

5

函数 `_filter_backtrack_doors()` `_is_entry_protected()` `_should_block_backtrack_direction()`


```

def _filter_backtrack_doors(self, doors: Sequence[BoxDict], direction_hint:
Optional[str]) -> List[BoxDict]:
    if not doors:
        return []

    if not Game._room_entry_position:
        return list(doors)

    now = time.time()
    protect_entry = now - Game._room_entry_time <= 3.0
    if not protect_entry:
        return list(doors)

    entry = Game._room_entry_position
    entry_side = self._reverse_direction(Game._flow_direction)
    filtered: List[BoxDict] = []
    for door in doors:
        x, y, w, h = door["xywh"]
        center = (x + w / 2.0, y + h / 2.0)
        near_entry = self._distance(center, entry) <= 240
        dx = center[0] - entry[0]
        dy = center[1] - entry[1]
        if abs(dx) >= abs(dy):
            door_side = "RIGHT" if dx > 0 else "LEFT"
        else:
            door_side = "DOWN" if dy > 0 else "UP"

        if not near_entry:
            filtered.append(door)
            continue

        if entry_side and door_side == entry_side:
            logger.info("ignore backtrack door at entry side {}: {}".format(
center)
                entry_side,
                door_side)
            continue

        reverse_hint = self._reverse_direction(direction_hint)
        if reverse_hint and door_side == reverse_hint:
            logger.info("ignore reverse door for hint {}: {}".format(
center)
                direction_hint,
                door_side)
            continue

        filtered.append(door)

    return filtered

def _is_entry_protected(self) -> bool:
    return bool(Game._room_entry_position) and (time.time() -
Game._room_entry_time <= 3.0)

```

```
def _should_block_backtrack_direction(self, direction: Optional[str]) -> bool:
    if not direction or not self._is_entry_protected():
        return False
    entry_side = self._reverse_direction(Game._flow_direction)
    if not entry_side:
        return False
    return direction.upper() == entry_side
```

段专 决 个很常 的 ： 刚 房， 没清怪，就因为 检测或 线判断抖动，立刻又 回上 房

滤 只在 房后的 3 秒保护期内生效 程序会拿每扇 和 “入口位 ” 比 ，先判断 扇是不是离入口很 只有离入口很 的 才会 点检查 如果 扇 刚好处在 “入口反方向的回头 ” ，就直接忽略；如果 扇 然不 定是入口 ，但它正好与当前 线提示方向相反，也会 忽略 后 的 `_should_block_backtrack_direction()` 则是在没有明确 可 时， 后备方向 做最后 回头检查

是 套 “抖” 和 “ 回头 ” 的启发式 则 它不复杂，但对刷图稳定性特别 ，因为很多 动化程序就是死在 繁掉头和原地来回穿 上

6

函数 `_key_press()` `_tap_direction()` `_reverse_direction()`

```
def _key_press(self, key: str) -> None:
    pydirectinput.keyDown(key)
    time.sleep(0.08)
    pydirectinput.keyUp(key)
    time.sleep(0.05)

def _tap_direction(self, direction: str, duration: float = 0.12) -> None:
    for action in direction.strip().split("_"):
        pydirectinput.keyDown(action.lower())
    time.sleep(duration)
    for action in direction.strip().split("_"):
        pydirectinput.keyUp(action.lower())
    time.sleep(0.04)

def _reverse_direction(self, direction: Optional[str]) -> Optional[str]:
    mapping = {
        "LEFT": "RIGHT",
        "RIGHT": "LEFT",
        "UP": "DOWN",
        "DOWN": "UP",
        "LEFT_UP": "RIGHT_DOWN",
        "LEFT_DOWN": "RIGHT_UP",
        "RIGHT_UP": "LEFT_DOWN",
        "RIGHT_DOWN": "LEFT_UP",
    }
    return mapping.get(direction or "")
```

封装了动作层最基础的操作：按下某个点某个方向以及取某个方向的反方向

`_key_press()` 合攻击 拾取 类短按操作，按下后等 0.08 秒再松开

`_tap_direction()` 可以处理单方向和斜方向，比如 `RIGHT_UP` 会拆成同时按右和上，持小段时间后松开 `_reverse_direction()` 则提供 8 个方向映射，后卡住恢复和回头滤镜会大使用它

如果把整套程序看成人在玩游戏，就是“手指动作库”上层只“按 x”“点右上”，底层就由些函数具体发出盘事件

函数

段 在 堆同类目标 , 比如 堆怪物或 堆 , 找出离玩家最 的 个

程序先确保 已知 玩家位 然后对每个目标, 把原始框换算成中心点, 与玩家中心做差得到 `dx` `dy` 如果传 来了方向 制 `direct` , 就只保留 方向上的目标, 比如期望往右时只保留 `dx > 0` 的目标 之后用欧氏 离算当前目标离玩家多 , 只 比目前最小 离小, 就更新为新的最 目标

战斗 拾取和 大 依 个函数 它体现的是 个很 素但实用的策略: 4优先处理离己最 最容易立刻交互到的目标

8

函数 `_get_direction()`

```
def _get_direction(self, obj_box: Sequence[float]) -> Optional[str]:
    if not self._player_xywh:
        return None

    dx = obj_box[0] - self._player_xywh[0]
    dy = obj_box[1] - self._player_xywh[1]

    if dy < 0:
        if abs(dy) < 20:
            return "RIGHT" if dx > 0 else "LEFT"
        if self._player_xywh[1] - obj_box[1] < abs(dx):
            return "RIGHT_UP" if dx > 0 else "LEFT_UP"
        return "UP"
    if dy > 0:
        if abs(dy) < 20:
            return "RIGHT" if dx > 0 else "LEFT"
        if dy < abs(dx):
            return "RIGHT_DOWN" if dx > 0 else "LEFT_DOWN"
        return "DOWN"
    return None
```

段代码 把“目标在玩家的哪 ” 成程序 执 的方向字符串，比如 `LEFT` `UP` `RIGHT_DOWN`

程序先 算目标中心相对玩家中心的横纵偏移 如果目标在玩家上方，先看上下差是否很小，很小就当成左右方向；否则再比 上下偏差和左右偏差，决定是“偏斜上方” 是“纯上方” 目标在下方时也是同样 的 `20` 像素 值就是 种容差，允 目标 然略点点，但仍然 当作左右方向处理

是动作层把空 关系变成按 方向的 器 后 的打怪 拾取 朝 移动，几乎 是先用它，再把 果扔 按 函数

9

函数 `_move()`

```
def _move(
    self,
    direction: str,
    is_slow: bool = False,
    _action_cache: Optional[str] = None,
    press_time: float = 0.1,
    release_time: float = 0.1,
) -> str:
    logger.info("current direction: {}, cached direction: {}", direction,
        _action_cache)
    if _action_cache and direction != _action_cache:
        for action in _action_cache.strip().split("_"):
            pydirectinput.keyUp(action.lower())

    for action in direction.strip().split("_"):
        pydirectinput.keyDown(action.lower())

    if not is_slow:
        time.sleep(press_time)
        for action in direction.strip().split("_"):
            pydirectinput.keyUp(action.lower())
        time.sleep(release_time)
        for action in direction.strip().split("_"):
            pydirectinput.keyDown(action.lower())

    return direction
```

就是动作层真正把“方向字符串” 成按 事件的核心函数

如果上 帧 存的方向和当前方向不同，就先把旧方向涉及的 松开，然后按下新方向对应的
如果不是慢 模式， 会执 次“按住 小段时 -> 松开 -> 再按下”的 奏， 样更
像 移动 最后 回当前方向，供外 更新到 `_action_cache`

你可以把它理 成动作控制层的“总 ” 所有“朝怪 ” “朝 ” “后备 线移动”
等操作，最后 会 到 个函数上

10

函数 `_update_motion_state()`

```
def _update_motion_state(self, expect_motion: bool, player_visible: bool) -> bool:
    if not self._player_xywh:
        return False
    if not expect_motion or not player_visible:
        Game._position_history.clear()
        Game._stuck_count = 0
        return False

    current = (self._player_xywh[0], self._player_xywh[1])
    Game._position_history.append(current)
    if len(Game._position_history) > 8:
        Game._position_history.pop(0)

    if len(Game._position_history) < 6:
        return False

    xs = [point[0] for point in Game._position_history]
    ys = [point[1] for point in Game._position_history]
    spread_x = max(xs) - min(xs)
    spread_y = max(ys) - min(ys)
    if spread_x < 18 and spread_y < 18:
        Game._stuck_count += 1
    else:
        Game._stuck_count = 0

    return Game._stuck_count >= 2
```

段代码专 判断：“程序明明在尝 移动，但 是不是其实几乎没动？”如果是，就把它为卡住

只有在两个条件 满 时才会做卡住检测：第 ，程序当前确实在期待 动；第二，玩家目标本 确实 检测到了 然后它会 录最 几帧玩家中心点位 ，最多保留 8 个 如果历史点不够 6 个，先不下 等样本够了以后，程序 算 些点在 X 和 Y 方向的扩散 围 横 向和纵向扩散 小于 18 像素，就 为 基本没动，卡住 数加 1；否则就清 两次满 个条件，才正式判断为卡住

是 套很典型的“位 历史容差法” 它不 求程序精准知 碰撞发生了什么，只看 果：最 段时 位 是不是几乎没变

11

函数 `_recover_from_stuck()` 与 `_recover_missing_player()`

```
def _recover_from_stuck(self, reason: str) -> None:
    now = time.time()
    if now - Game._last_recover_time < 1.2:
        return

    try:
        Game._last_recover_time = now
        Game._stuck_count = 0
        Game._position_history.clear()
        logger.warning("stuck recovery triggered: {}", reason)
        current_hint = Game._action_cache or self._current_door_direction() or
Game._flow_direction or "RIGHT"
        self._release_cached_action()
        reverse_hint = self._reverse_direction(current_hint) or "LEFT"
        pattern = Game._recover_step % 4
        Game._recover_step += 1

        if pattern == 0:
            self._tap_direction(reverse_hint, 0.18)
            self._tap_direction("DOWN", 0.12)
        elif pattern == 1:
            self._tap_direction(reverse_hint, 0.18)
            self._tap_direction("UP", 0.12)
        elif pattern == 2:
            for step in ("RIGHT", "DOWN", "LEFT", "UP"):
                self._tap_direction(step, 0.1)
        else:
            self._tap_direction(reverse_hint, 0.22)
            self._tap_direction(current_hint if isinstance(current_hint, str) else
"RIGHT", 0.1)

        Game._action_cache = None
    except Exception as exc:
        logger.exception("stuck recovery failed: {}", exc)
        self._release_cached_action()
        Game._action_cache = None

def _recover_missing_player(self) -> None:
    if Game._player_missing_count < Game._missing_player_recover_threshold:
        return

    logger.warning("player lost for {} frames, try recovery",
Game._player_missing_count)
    self._recover_from_stuck("player_missing")
    Game._player_missing_count = 0
```

旦程序判断 已卡住, 或 很多帧 找不到玩家, 它就会 入恢复 , 尝 把 新从僵局 拽出来

先有 个恢复冷却, 免短时 反复乱按 然后清空卡住 数和位 历史, 并 择 个 “当前提示方向” 个提示方向的来源优先级是: 当前 存动作, 其次是 线方向, 再其次是房 流向, 实在不 往右 然后 算反方向, 按四种模式 流尝 困: 反方向再下 反方向再上 个小矩形 反方向再朝当前方向 探 `_recover_missing_player()` 则在 很多帧 丢失玩家检测 果时, 直接复用 套卡住恢复

是动作层的 “ 系 ” 因为 动化 总会 到碰撞 怪物挡 别抖动等 , 如果没有恢复策略, 程序很容易永 卡在某个

12

函数 `_kill_monster()`

```
def _kill_monster(self, obj_box: Sequence[float]) -> None:
    if not self._player_xywh:
        return

    if abs(obj_box[0] - self._player_xywh[0]) < self._attack_x and abs(obj_box[1]
- self._player_xywh[1]) < self._attack_y:
        direction = self._get_direction(obj_box)
        face = None
        if direction:
            for item in direction.split("_"):
                if "RIGHT" in item:
                    face = "RIGHT"
                elif "LEFT" in item:
                    face = "LEFT"
                break

        if face:
            self._key_press(face.lower())
            self._key_press("x")
            self._release_cached_action()
            return

        direction = self._get_direction(obj_box)
        if direction:
            Game._action_cache = self._move(direction, is_slow=True,
_action_cache=Game._action_cache)
```

段代码 处理 “我 去打 个怪” 的动作 常直接：够 就打，不够 就 去

程序先看怪物中心和玩家中心在 X Y 方向上的差值，是否 入了攻击 围 值 `_attack_x` 和 `_attack_y` 如果 入 围，就先根据方向决定 朝左 是朝右 ， 然后按 下 `x` 攻击，再把之前 存的移动动作 放掉 如果 没 入攻击 围，就 算怪物方向， 用 `_move(..., is_slow=True)` 慢慢朝怪物

明当前战斗不是复杂技 招， 是 套 “接 目标 -> 向目标 -> 按攻击 ” 的基础打法 对小白来 ， 正好很容易 懂：程序其实就是在模仿最简单的手动打怪流程

13

函数 `_pick_up()` 与 `_move_to_door()`

```
def _pick_up(self, obj_box: Sequence[float]) -> None:
    if not self._player_xywh:
        return

    if abs(obj_box[0] - self._player_xywh[0]) < self._move_x and abs(obj_box[1] -
self._player_xywh[1]) < self._move_y:
        self._key_press("f")
        self._release_cached_action()
        return

    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction, is_slow=True,
_action_cache=Game._action_cache)

def _move_to_door(self, obj_box: Sequence[float]) -> None:
    direction = self._get_direction(obj_box)
    if direction:
        Game._action_cache = self._move(direction,
_action_cache=Game._action_cache)
```

段代码分别 两件事：捡掉 物，以及朝某 扇 移动

`_pick_up()` 的 和打怪很像：如果已 入拾取 围，就按 `f`，并 放掉之前的移动动作；否则就朝物品方向慢慢 `_move_to_door()` 更简单，它先 算 的方向，然后 用 `_move()` 朝 去

两个函数 明当前系 的 为控制其实是 的：对怪 对物品 对 ，本 是“先算相对方向，再决定是交互 是 ”

14

函数 `_reset_route_search()` 与 `_fallback_up_search_move()`

```
def _reset_route_search(self) -> None:
    Game._route_search_direction = None
    Game._route_search_phase = 0
    Game._route_search_start = None

def _fallback_up_search_move(self) -> None:
    if not self._player_xywh:
        Game._action_cache = self._move("UP", is_slow=True,
        _action_cache=Game._action_cache)
        return

    current = (self._player_xywh[0], self._player_xywh[1])
    if Game._route_search_direction != "UP":
        Game._route_search_direction = "UP"
        Game._route_search_phase = 0
        Game._route_search_start = current

    if Game._route_search_start is None:
        Game._route_search_start = current

    start_x, start_y = Game._route_search_start
    current_x, current_y = current
    phase = Game._route_search_phase
    amplitude = 20.0

    if phase == 0:
        command = "DOWN"
        if current_y - start_y >= amplitude:
            Game._route_search_phase = 1
            Game._route_search_start = current
            command = "RIGHT"
    elif phase == 1:
        command = "RIGHT"
        if current_x - start_x >= amplitude:
            Game._route_search_phase = 2
            Game._route_search_start = current
            command = "UP"
    elif phase == 2:
        command = "UP"
        if start_y - current_y >= amplitude:
            Game._route_search_phase = 3
            Game._route_search_start = current
            command = "RIGHT"
    else:
        command = "RIGHT"
        if current_x - start_x >= amplitude:
            Game._route_search_phase = 0
            Game._route_search_start = current
            command = "DOWN"
```

```
logger.info(
    "up fallback sweep: phase={}, start={}, current={}, command={}",
    Game._route_search_phase,
    Game._route_search_start,
    current,
    command,
)
Game._action_cache = self._move(command, is_slow=True,
    _action_cache=Game._action_cache)
```

段代码是个很有意思的特例：当后备 线提示 求向上，但当前又没检测到清晰 目标时，程序不会 直往上 ， 是执 套扫图式搜索动作

程序把 个搜索 程看成 个四 段小状态机 每 段 会朝 个固定方向慢慢 ， 直到位移 `amplitude = 20` 像素，就切到下个 段 整体 大 是“先向下 点，再向右，再向上，再向右”，形成 种小 围扫描动作 样做是为了在没有明显上 时，尝 移动到更容易看到或 入上 的位

不是严格意义上的寻 算法， 是 种启发式搜 手法 它反映了实 刷图中的 ：某些 尤其是上 ， 必 站到特定位 才 利接 ， 所以程序会主动扫 下位

15

函数 `_fallback_route_move()` 与 `_check_position()`

```
def _fallback_route_move(self, direction: str) -> None:
    normalized = direction.upper()
    if normalized == "DOWN":
        self._reset_route_search()
        if not self._player_xywh:
            Game._action_cache = self._move("DOWN", is_slow=True,
            _action_cache=Game._action_cache)
            return
        player_x = self._player_xywh[0]
        if player_x > self._width * 0.62:
            normalized = "LEFT_DOWN"
        elif player_x < self._width * 0.38:
            normalized = "RIGHT_DOWN"
    elif normalized == "UP":
        self._fallback_up_search_move()
        return
    else:
        self._reset_route_search()

    Game._action_cache = self._move(normalized, is_slow=True,
    _action_cache=Game._action_cache)

def _check_position(self) -> str:
    if not self._player_xywh:
        return "RIGHT_DOWN"
    player_x = self._player_xywh[0]
    player_y = self._player_xywh[1]

    if player_x < self._width / 2:
        return "LEFT_UP" if player_y < self._height / 2 else "LEFT_DOWN"
    return "RIGHT_UP" if player_y < self._height / 2 else "RIGHT_DOWN"
```


当前画 如果没有明确目标，比如怪 物品 不明显，但导 器又 了 个 线方向，
段代码 根据 个方向做 个 底移动

如果后备方向是下，程序会根据玩家在屏幕横向位 ， 当改成左下或右下， 样更容易撞到真正的下 方向是上，则交 前 的“向上扫 模式” 如果方向是左右，则直接慢 朝 方向移动 `_check_position()` 则根据玩家中心 在屏幕哪个 ， 回 个组合方向字符串，后 某些多步方向列 会用它来判断当前是否已 偏到某个区域

是没有明确目标时的 “ 位指南” 它并不 明，但 够实用： 少保 程序在没有 框或 别不稳时，不会完全原地发

16

函数 `run()` 的前半段

```
def run(self) -> None:
    raw_player = self._get_cls("player")
    if raw_player is None:
        Game._player_missing_count += 1
        Game._player = Game._pre_player
    else:
        Game._player_missing_count = 0
        Game._player = raw_player
        Game._pre_player = raw_player

    if raw_player is None and Game._player_missing_count >=
Game._missing_player_recover_threshold:
        self._recover_missing_player()
        return

    if Game._player is None:
        return
```

是 `Game.run()` 开始决策前的准备工作，核心是在确 玩家当前有没有 检测出来

程序先尝 从检测对 取出当前玩家框 如果 帧没找到玩家，就把 “ 丢失玩家帧数” 加 ，并且临时沿用 上 帧的玩家位 `_pre_player` ， 样可以减少偶发漏检带来的抖动 如果 帧成功找到玩家，就把 数清 ，并更新当前玩家和上 帧玩家 存 如果丢失太多帧，就 发前 的恢复 ；如果 上 帧 存也没有， 帧就什么 不做

动作层任何判断 离不开玩家位 ，所以 相当于先 己“定位” 且它 到了 YOLO 漏检的现实情况，不会因为 帧没 出玩家就立刻全线失控

17

函数 `run()` 的中段

```
try:
    self._player_xywh = list(Game._player["xywh"])
    self._player_xywh[0] = self._player_xywh[0] + self._player_xywh[2] / 2
    self._player_xywh[1] = self._player_xywh[1] + self._player_xywh[3] / 2
    logger.info("player center: {}", self._player_xywh[:2])
    self._maybe_update_room_entry()

    if self._update_motion_state(
        expect_motion=Game._action_cache is not None,
        player_visible=raw_player is not None,
    ):
        self._recover_from_stuck("movement_stalled")
        return
```

开始把玩家框换算成玩家中心点，并立刻做“房 入口更新”和“卡住检测”

检测器 回的玩家框格式是左上 加宽 ，所以 先把它 换成中心坐标 然后打印日志，接着用 个中心点去更新房 入口信息 做完 些后，再判断当前是不是处于“程序在移动，但 实 上几乎没动”的卡住状态 如果卡住，立刻执 恢复 并 束 帧，不再往下做打怪或 判断

步体现了 个很合理的优先级：如果 已 卡住， 无 你后 打怪 是 ，先把卡住 决掉才有意义

18

函数 `run()` 的中段二

```
boss = self._get_cls("boss")
if boss:
    nearest_boss = self._get_nearest(boss)
    if nearest_boss:
        self._kill_monster(nearest_boss["xywh"])
        return

monsters = self._get_cls("monster")
if monsters:
    nearest_monster = self._get_nearest(monsters)
    if nearest_monster:
        self._kill_monster(nearest_monster["xywh"])
        return
```

段定义了战斗优先级：如果画 有 Boss，先打 Boss；没有 Boss，才去打普 怪

程序先把所有 Boss 检出来，只 存在，就找最 的 Boss 并 用 `_kill_monster()` 如果 帧已 入打 Boss 分支，就直接 `return`，不再 判断物品和 没有 Boss，再对 普 怪做相同处理

明动作层是 个 “ 优先 截断” 的 也就是 ， 帧只做 件最 的事，不会 打怪 又尝 去捡东 或

19

函数 `run()` 的中段三

```
goods = self._get_class("goods")
if goods:
    nearest_goods = self._get_nearest(goods)
    if nearest_goods:
        self._pick_up(nearest_goods["xywh"])
    return

money = self._get_class("money")
if money:
    nearest_money = self._get_nearest(money)
    if nearest_money:
        self._pick_up(nearest_money["xywh"])
    return
```

当 帧已 没有 Boss 和怪物 处理时，程序会开始 拾取地上的物品和 币

它先查看是否有 掉 物 `goods`，有的 找最 的并 用 `_pick_up()` 如果没有普 掉 物，再看是否有 币 `money` 同样， 旦 入拾取分支并执 ，当前 帧就 束，不会 往下判断

体现出刷图 为的整体优先 序：先清战斗目标，再拾取奖励，最后才 推

20

函数 `run()` 的 处理 分

```

doors = self._get_class("door")
if doors:
    direction_hint = self._current_door_direction()
    if isinstance(self._direction, (list, tuple)) and Game._index > 0:
        position = self._check_position()
        for item in position.split("_"):
            if direction_hint == item:
                Game._index += 1
                direction_hint = self._current_door_direction()

    usable_doors = self._filter_backtrack_doors(doors, direction_hint)
    nearest_door = self._get_nearest(usable_doors, direct=direction_hint)
if direction_hint else None
    if nearest_door is None and direction_hint:
        logger.info("doors found but none match route hint {}, keep
following hint", direction_hint)
        raise LookupError("no door matches route hint")
    if nearest_door is None and usable_doors:
        nearest_door = self._get_nearest(usable_doors)
if nearest_door:
    self._move_to_door(nearest_door["xywh"])
return

```

段专 “清完当前房 之后, 按照 线方向去找 并往 ”

程序先拿到所有 , 再取当前 线方向提示 如果方向本 是个列 , 它 会 合当前玩家所在屏幕 , 尝 把多步方向列 向前推 接着先用前 的入口保护 滤掉明显的回头 , 再根据方向提示只找 方向上的最 如果明明有方向提示, 却完全找不到匹 方向的 , 就抛出 个 `LookupError` , 示当前不按 , 是把控制权交 后 的底 如果没有严格匹 的 但 有可用 , 则 求其次, 最 的可用 最 只 拿到 , 就 用 `_move_to_door()` 朝它

是房 级 径真正 地到当前画 为的关 段 A 星 的是抽 方向, 把抽 方向 实成 “当前 扇 ”

21

函数 `run()` 的 尾

```
except LookupError:
    pass
except Exception as exc:
    logger.exception("game loop error: {}", exc)
    self._release_cached_action()
    return

fallback_direction = self._current_door_direction()
if fallback_direction:
    if self._should_block_backtrack_direction(fallback_direction):
        logger.warning("fallback direction {} points to entry side, hold position", fallback_direction)
        self._release_cached_action()
        return
    logger.warning("no target found, fallback move: {}", fallback_direction)
    self._fallback_route_move(fallback_direction)
    return

logger.warning("no target found and no route hint, hold position")
self._release_cached_action()
```

段代码 在前 所有主分支 没成功时，决定程序最后 怎么办：是 一个后备方向，是干 原地停住

如果前 找 时故意抛出了 `LookupError`， 不会当成严重 异常， 是 底 只有真正未 料到的异常才会 录日志并 放动作 之后，它再去看有没有 线方向提示 如果有，就先检查 个方向是不是明显指向入口回头 ；如果是，就宁可停住也不乱 否则就用 `_fallback_route_move()` 做 个带启发式的后备移动 如果 线提示也没有，就 录 告并 放按 ，保持原地等待

段体现的是动作系 的最后 安全 动化程序最 “没想清楚也硬动”， 的 更 慎：找不到明确目标时，优先做受控的后备动作，实在不 就停

minimap_route_demo.py

它不是主刷图 本， 是 个单独的测 本，用来 “截图 + 大画
检测 + 小地图导 ” 条是否工作正常

1

文件前半段

```
import json
import time

import cv2
import numpy as np
import pyautogui

from dnf.detector import Detector
from dnf.minimap_nav import MiniMapNavigator

def main() -> None:
    width = 1067
    height = 600
    detector = Detector("")
    navigator = MiniMapNavigator(map_name="generic")

    while True:
        started = time.time()
        img = pyautogui.screenshot(region=(0, 0, width, height))
        frame_rgb = np.array(img)
        annotated, objects = detector.detect(frame_rgb)
        frame_bgr = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)
```

段 本做的事情很像 `main.py` 的前半段，但它不是 定某个游戏窗口， 是直接截屏幕固定区域，然后 检测和导

先初始化检测器和导 器，然后在循环 每次截屏左上 固定尺寸区域，把截图 换成数组，再分别 YOLO 检测器和 OpenCV 导 器

它主 是个 工具，帮助开发 在不真正 入完整刷图主循环的情况下，快 导 是否正常

2

文件后半段

```
snapshot = navigator.build_route_snapshot(frame_bgr, objects or [])
print(
    json.dumps(
        {
            "current_room": snapshot.current_room,
            "boss_room": snapshot.boss_room,
            "query_room": snapshot.query_room,
            "elite_room": snapshot.elite_room,
            "next_room_direction": snapshot.next_room_direction,
            "selected_door_center": snapshot.selected_door_center,
            "cost_ms": round((time.time() - started) * 1000, 1),
        },
        ensure_ascii=False,
    )
)

if annotated is not None:
    minimap = navigator.extract_minimap(frame_bgr)
    cv2.imshow("dnf-route-frame", cv2.resize(annotated, (800, 450)))
    cv2.imshow("dnf-route-minimap", cv2.resize(minimap, (324, 180)))

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

段代码把导 器的 果打印出来，并把检测图和小地图可 化显示 人看

程序 用 `build_route_snapshot()` 后，把当前房 Boss 房 号房 下 步方向
中的 中心和 时 打印成 JSON 然后如果检测图不为空，就显示两扇窗口： 扇是整张大
画 的检测 果， 扇是单独 出来的小地图 按 `q` 时 出

就是 个 “ 由体检工具” 它不 按 不 刷图，只 你 眼 导 层
到底准不准

olaplug

`olaplug` 目录 主 是个 常大的 DLL 封 层, 包含窗口 定 入 图像 图 OCR YOLO 接口 内存接口等大 用函数 但当前 套刷图主流程并没有真正实例化 `OLAPPlugServer`, 也没有 用它的 `BindWindow()`

olaplug

你现在最 知 的, 不是它的每个函数, 是它有没有接入主流程

主刷图当前真正走的链路:

`win32gui` 找窗口

- > `pyautogui` 截图
- > `detector.py` 做 YOLO 识别
- > `minimap_nav.py` 识别小地图并调用 A 星
- > `game.py` 用 `pydirectinput` 发方向键 / 攻击键 / 拾取键

而不是:

`OLAPPlugServer.BindWindow()`

- > 后台绑定
- > OLA 输入接口

段是为了 止你 代码时 偏 `olaplug` 文件很大, 接口极多, 如果 上来就 去, 会 常容易把主线看丢

当前主刷图代码 真正 到的, 是 Windows 找窗口 截图 检测 导 和按 执 条 `olaplug` 更像 个备用工具箱, 或 旧版本/扩展版本可 会使用的底层 力库

如果你之后想深入研究“更底层的后台 定和 DLL 力”, 我可以下 专 单独 你做 本 `olaplug` 明书 但就“当前刷图代码从头到尾怎么 ” 个目标来 , 前 六章才是你最 抓住的主线

你现在如果按 序 新回看源码, 已 不 再把它看成 大 “完全看不懂的代码”了 你可以用 句 串 来:

A

文档生成时 : 2026-04-23 文档基于当前工作区 `D:\yolo\yolo11\dnf` 的源码内容整理